

Practical 9

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#define BUFFER_SIZE 4096
void low_level_copy(const char *source, const char *destination)
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;
    src = open(source, O_RDONLY);
    if (src == -1)
    {
        perror("Error opening source file");
        return;
    }
    dest = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest == -1)
    {
        perror("Error opening destination file");
        close(src);
        return;
    }
    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);
        if (bytes_written != bytes_read)
```

```

        if (bytes_written != bytes_read)
        {
            perror("Error writing file");
            break;
        }
    }
    if (bytes_read == -1)
        perror("Error reading file");
    close(src);
    close(dest);
}

void standard_library_copy(const char *source, const char *destination)
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;
    src = fopen(source, "rb");
    if (src == NULL)
    {
        perror("Error opening source file");
        return;
    }
    dest = fopen(destination, "wb");
    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        return;
    }
}

```

```

        return;
    }
    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        fwrite(buffer, 1, bytes_read, dest);
    }
    fclose(src);
    fclose(dest);
}

int main()
{
    char source[100];
    char low_dest[100];
    char std_dest[100];
    clock_t start, end;
    double low_time, std_time;
    printf("Enter source file: ");
    scanf("%99s", source);
    printf("Enter low-level destination file: ");
    scanf("%99s", low_dest);
    printf("Enter standard library destination file: ");
    scanf("%99s", std_dest);
    printf("\n--- Low-Level File Copy ---\n");
    start = clock();
    low_level_copy(source, low_dest);
    end = clock();
    low_time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Low-level copy completed.\n");
    printf("Time taken: %.6f seconds\n", low_time);
}

```

```

    start = clock();
    low_level_copy(source, low_dest);
    end = clock();
    low_time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Low-level copy completed.\n");
    printf("Time taken: %.6f seconds\n", low_time);
    printf("\n--- Standard Library File Copy ---\n");
    start = clock();
    standard_library_copy(source, std_dest);
    end = clock();
    std_time = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Standard library copy completed.\n");
    printf("Time taken: %.6f seconds\n", std_time);
    printf("\n--- Performance Comparison ---\n");
    printf("Low-level I/O time      : %.6f seconds\n", low_time);
    printf("Standard library I/O time: %.6f seconds\n", std_time);
    return 0;
}

```

```

root@tejasri:~# nano filecopy.c
root@Tejasri:~# gcc filecopy.c -o filecopy
root@Tejasri:~# ./filecopy
Enter source file: input.txt
Enter low-level destination file: lowcopy.txt
Enter standard library destination file: stdcopy.txt

--- Low-Level File Copy ---
Error opening source file: No such file or directory
Low-level copy completed.
Time taken: 0.000165 seconds

--- Standard Library File Copy ---
Error opening source file: No such file or directory
Standard library copy completed.
Time taken: 0.000075 seconds

--- Performance Comparison ---
Low-level I/O time      : 0.000165 seconds
Standard library I/O time: 0.000075 seconds
root@Tejasri:~#

```

Task 2

Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

```

GNU nano 7.2                                     redirection.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
int main()
{
    int input_fd, output_fd;
    input_fd = open("input.txt", O_RDONLY);
    if (input_fd == -1)
    {
        perror("Error opening input.txt");
        return 1;
    }
    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_fd == -1)
    {
        perror("Error opening output.txt");
        close(input_fd);
        return 1;
    }
    if (dup2(input_fd, STDIN_FILENO) == -1)
    {
        perror("Error redirecting stdin");
        close(input_fd);
        close(output_fd);
        return 1;
    }
    if (dup2(output_fd, STDOUT_FILENO) == -1)
    {

```

```

    }
    if (dup2(output_fd, STDOUT_FILENO) == -1)
    {
        perror("Error redirecting stdout");
        close(input_fd);
        close(output_fd);
        return 1;
    }
    close(input_fd);
    close(output_fd);
    printf("This message is redirected to output.txt.\n");
    char buffer[100];
    if (fgets(buffer, sizeof(buffer), stdin) != NULL)
    {
        printf("Data read from input.txt: %s", buffer);
    }
    return 0;
}

```

Output

```

root@Tejasri:~# nano redirection.c
root@Tejasri:~# gcc redirection.c -o redirection
root@Tejasri:~# ./redirection
Output redirected to output.txt
root@Tejasri:~# |

```